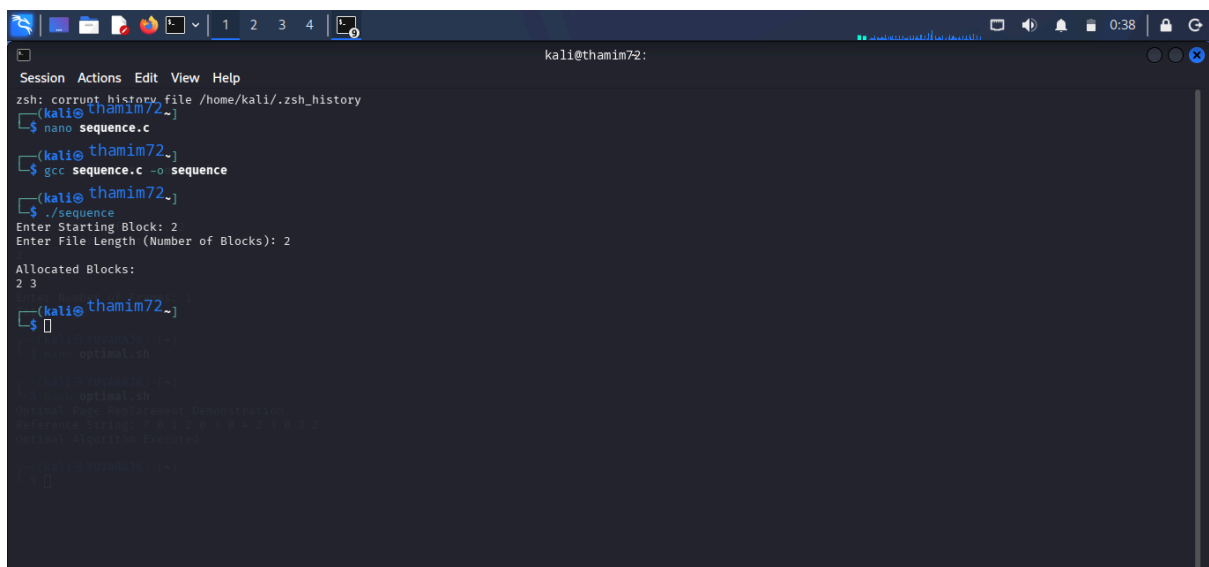


C PROGRAMMING :SEQUENTIAL



```
GNU nano 9.0 sequence.c
#include <stdio.h>
int main()
{
    int start, length, i;
    printf("Enter Starting Block: ");
    scanf("%d", &start);
    printf("Enter File Length (Number of Blocks): ");
    scanf("%d", &length);
    printf("\nAllocated Blocks:\n");
    for(i = 0; i < length; i++)
    {
        printf("%d ", start + i);
    }
    printf("\n");
    return 0;
}
```

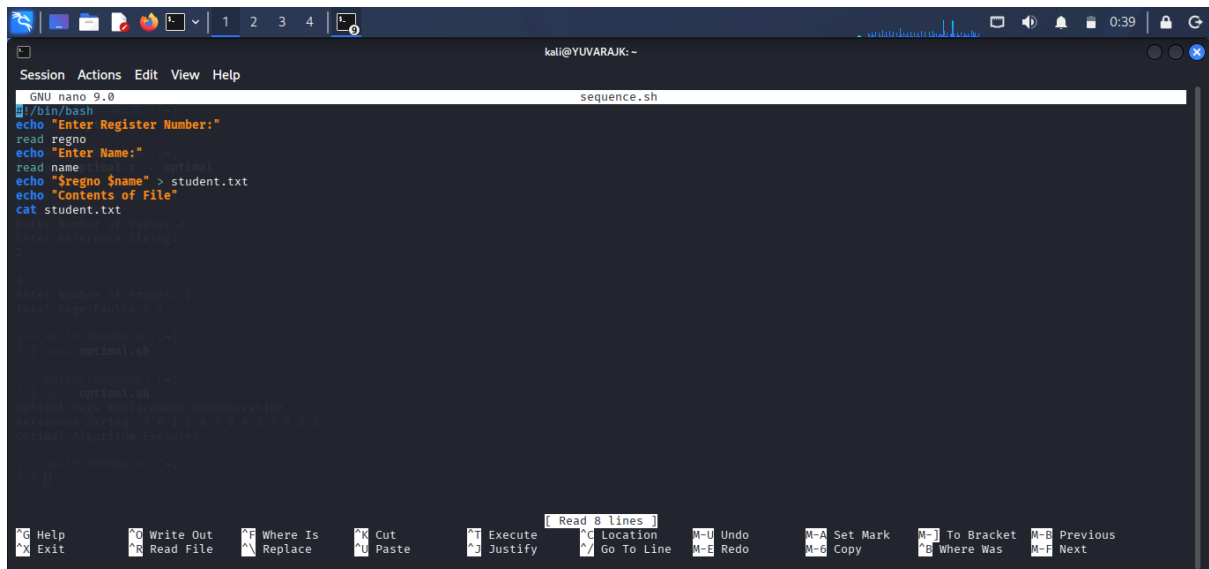
OUTPUT :



```
kali@thamim72:~$ zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72~)$ nano sequence.c
(kali@thamim72~)$ gcc sequence.c -o sequence
(kali@thamim72~)$ ./sequence
Enter Starting Block: 2
Enter File Length (Number of Blocks): 2

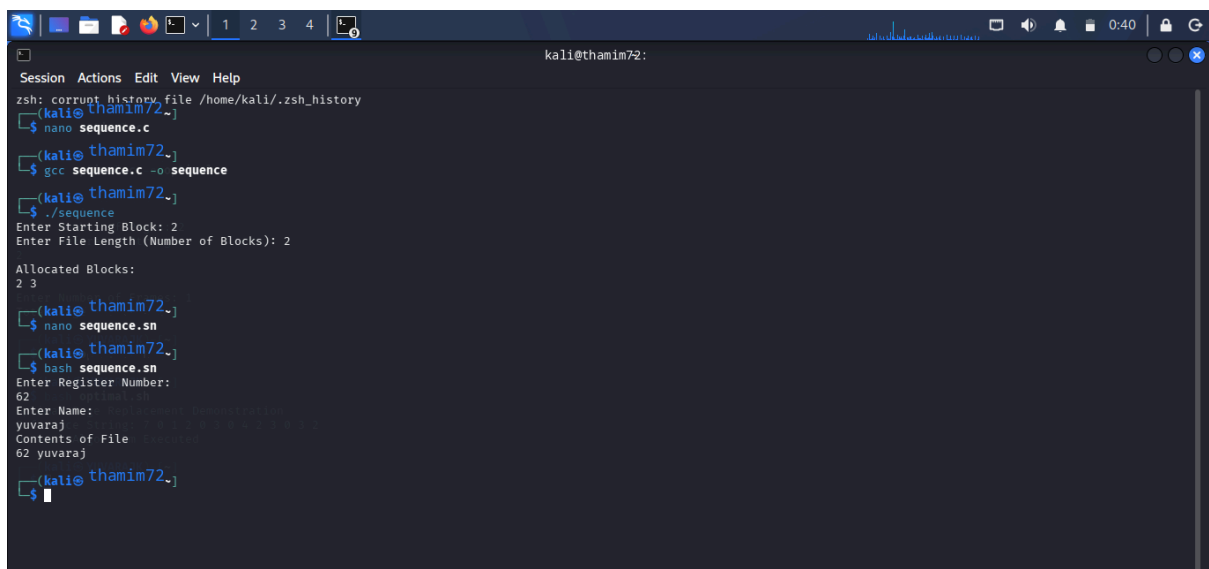
Allocated Blocks:
2 3
(kali@thamim72~)$
```

SHELL PROGRAMMING :SEQUENTIAL



```
GNU nano 9.0 sequence.sh
#!/bin/bash
echo "Enter Register Number:"
read regno
echo "Enter Name:"
read name
echo "$regno $name" > student.txt
echo "Contents of File"
cat student.txt
```

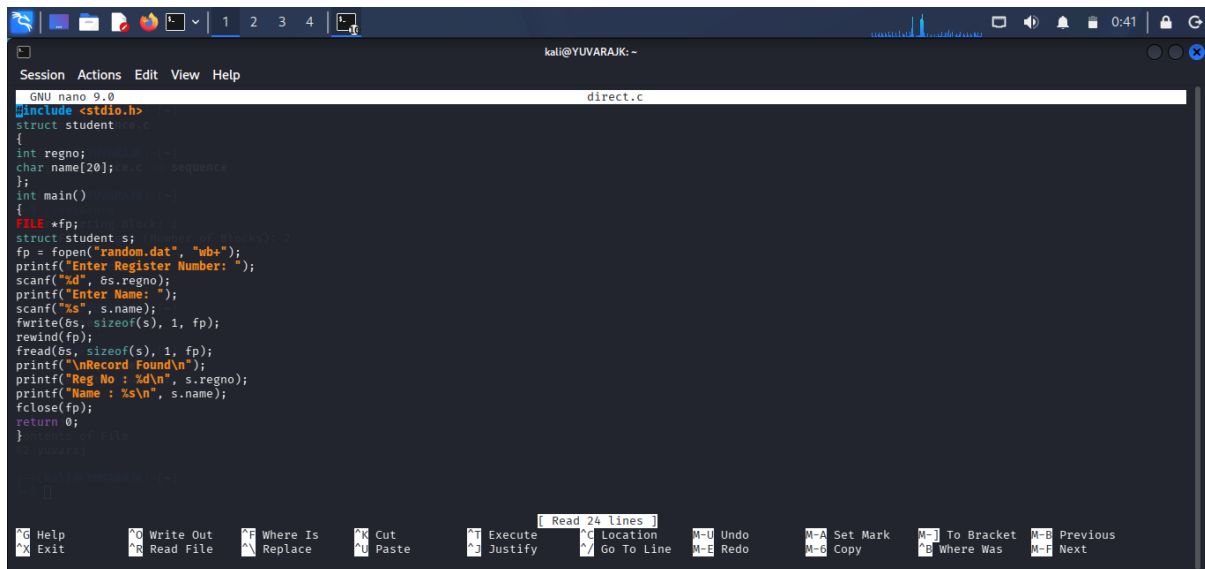
OUTPUT :



```
zsh: current history file /home/kali/.zsh_history
(kali@thamim72~)
$ nano sequence.c
(kali@thamim72~)
$ gcc sequence.c -o sequence
(kali@thamim72~)
$ ./sequence
Enter Starting Block: 2
Enter File Length (Number of Blocks): 2

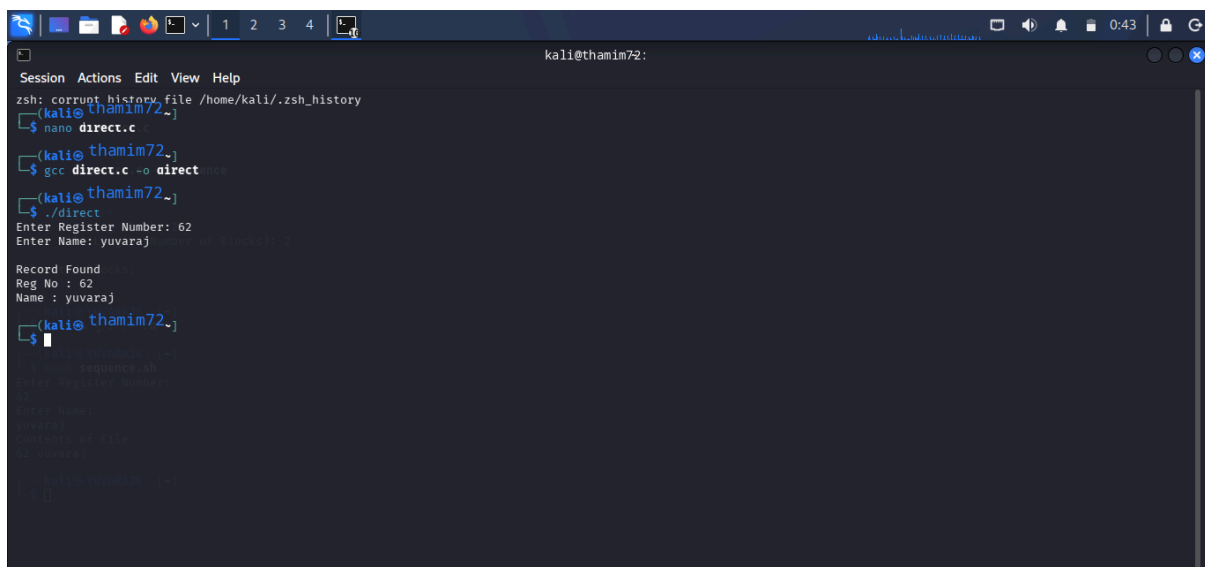
Allocated Blocks:
2 3
(kali@thamim72~)
$ nano sequence.sn
(kali@thamim72~)
$ bash sequence.sn
Enter Register Number:
62
Enter Name:
yuvaraj
Contents of File
62 yuvaraj
(kali@thamim72~)
$
```

C PROGRAMMING :DIRECT



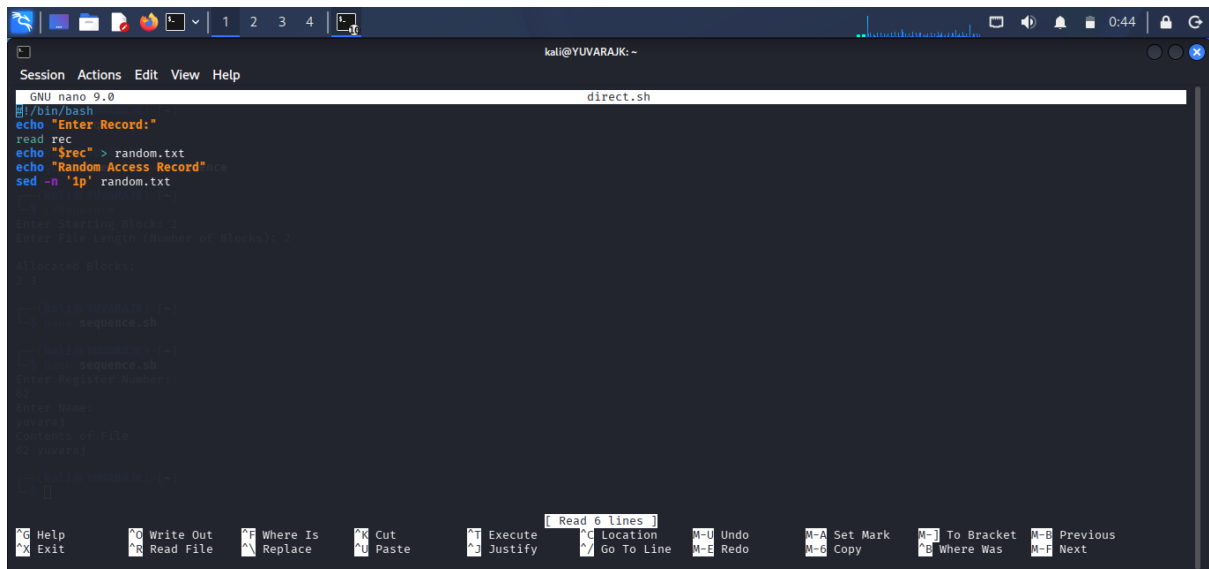
```
GNU nano 9.0 direct.c
#include <stdio.h>
struct student {
    int regno;
    char name[20];
};
int main()
{
    FILE *fp;
    struct student s;
    fp = fopen("random.dat", "wb+");
    printf("Enter Register Number: ");
    scanf("%d", &s.regno);
    printf("Enter Name: ");
    scanf("%s", s.name);
    fwrite(&s, sizeof(s), 1, fp);
    rewind(fp);
    fread(&s, sizeof(s), 1, fp);
    printf("\nRecord Found\n");
    printf("Reg No : %d\n", s.regno);
    printf("Name : %s\n", s.name);
    fclose(fp);
    return 0;
}
```

OUTPUT :



```
kali@thamim72:~$ nano direct.c
kali@thamim72:~$ gcc direct.c -o direct
kali@thamim72:~$ ./direct
Enter Register Number: 62
Enter Name: yuvaraj
Record Found
Reg No : 62
Name : yuvaraj
kali@thamim72:~$
```

SHELL PROGRAMMING :DIRECT

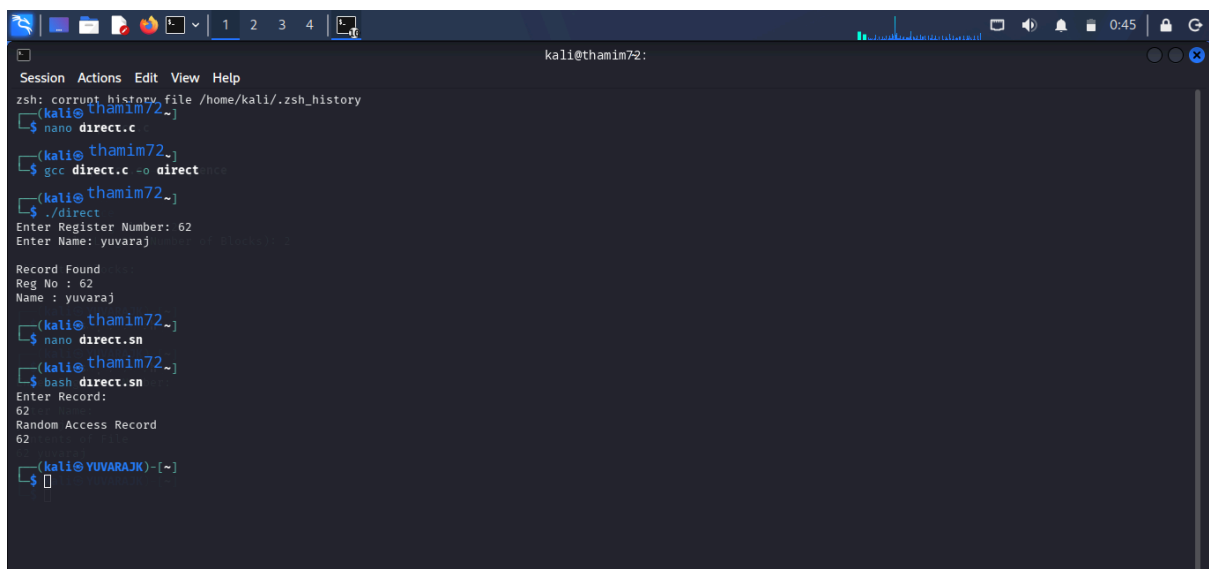


The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named 'direct.sh'. The script content is as follows:

```
#!/bin/bash
echo "Enter Record:"
read rec
echo "$rec" > random.txt
echo "Random Access Record"
sed -n '1p' random.txt
```

The nano editor interface includes a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The status bar at the bottom shows various keyboard shortcuts for editing and navigation.

OUTPUT :

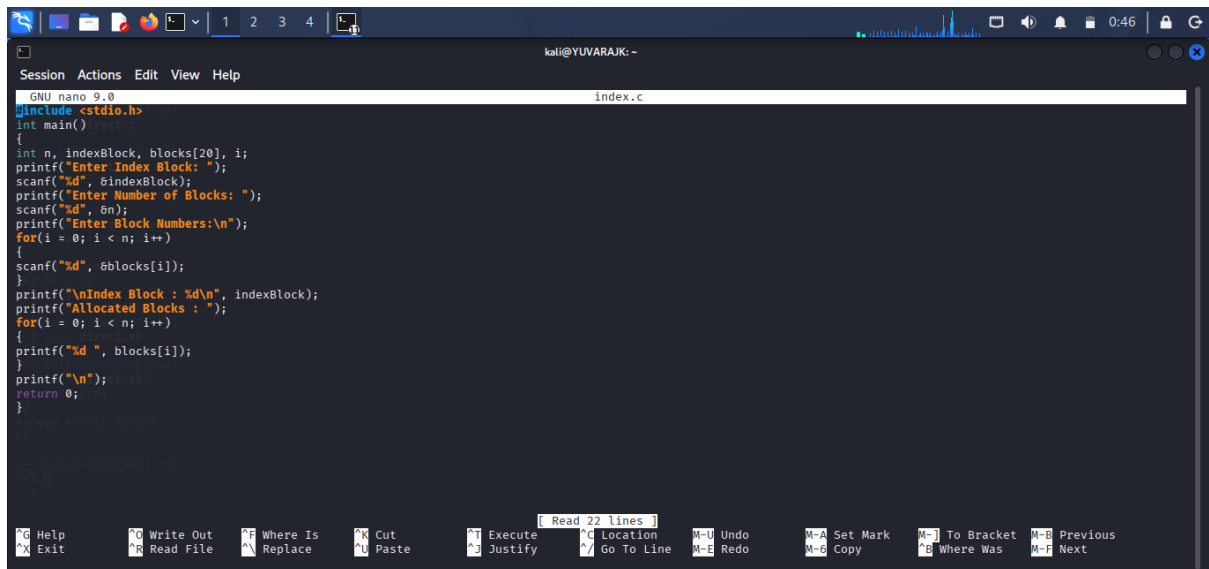


The screenshot shows a terminal window with the following commands and output:

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72~)
$ nano direct.c
(kali@thamim72~)
$ gcc direct.c -o direct
(kali@thamim72~)
$ ./direct
Enter Register Number: 62
Enter Name: yuvaraj
Record Found
Reg No : 62
Name : yuvaraj
(kali@thamim72~)
$ nano direct.sn
(kali@thamim72~)
$ bash direct.sn
Enter Record:
62
Random Access Record
62
(kali@YUVARAJK)-[~]
$
```

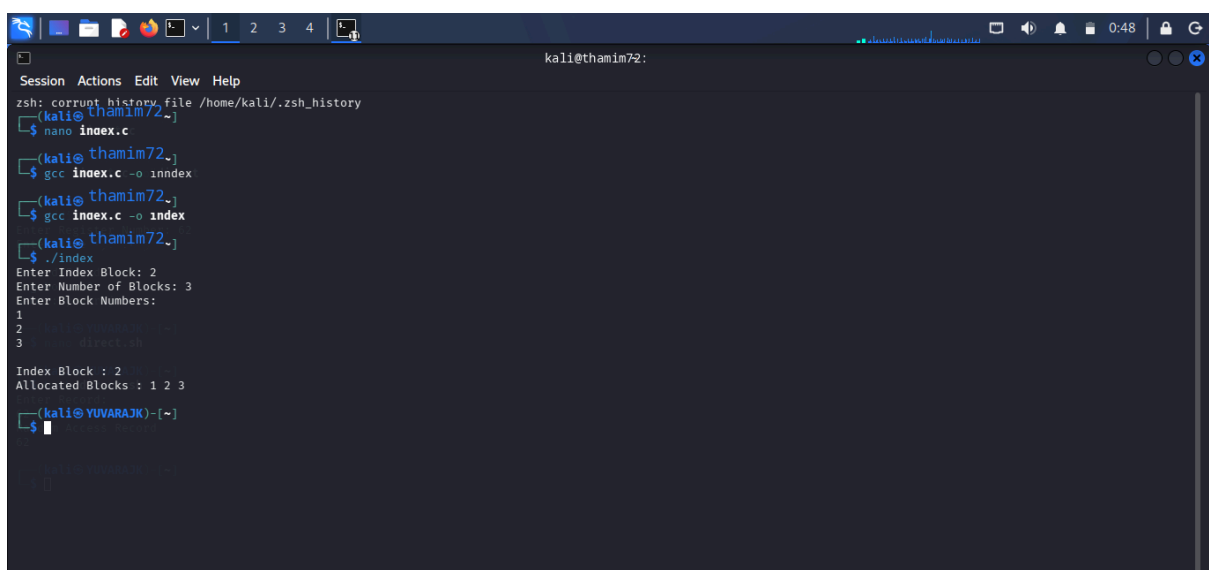
The terminal shows the user navigating through the nano editor to create 'direct.c', compiling it to 'direct', and then running it. The script prompts for a register number and a name, finds a record, and then prompts for a new record to be added to the 'random.txt' file.

C PROGRAMMING :INDEX



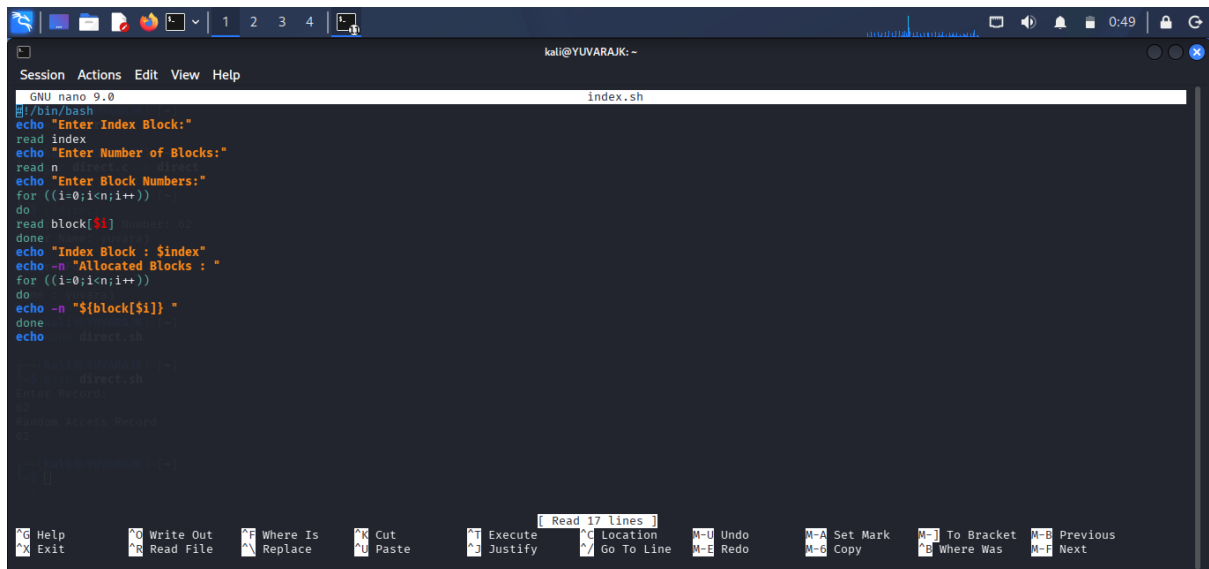
```
GNU nano 9.0 index.c
#include <stdio.h>
int main()
{
    int n, indexBlock, blocks[20], i;
    printf("Enter Index Block: ");
    scanf("%d", &indexBlock);
    printf("Enter Number of Blocks: ");
    scanf("%d", &n);
    printf("Enter Block Numbers:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &blocks[i]);
    }
    printf("\nIndex Block : %d\n", indexBlock);
    printf("Allocated Blocks : ");
    for(i = 0; i < n; i++)
    {
        printf("%d ", blocks[i]);
    }
    printf("\n");
    return 0;
}
```

OUTPUT :



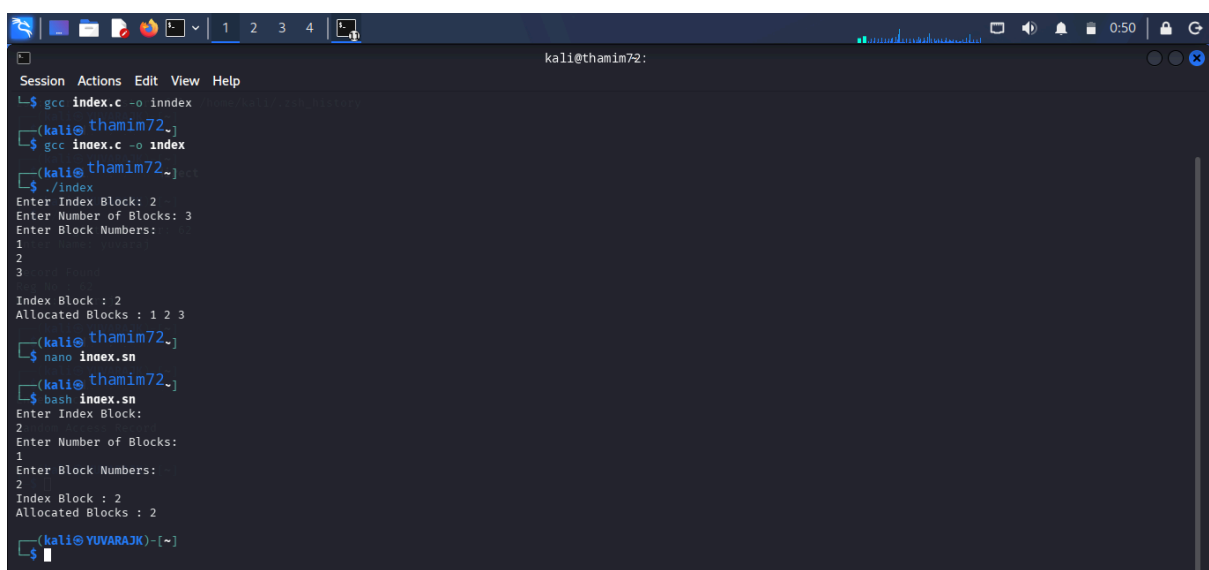
```
kali@thamim72:~$ nano index.c
(kali@thamim72~)$ gcc index.c -o index
(kali@thamim72~)$ ./index
Enter Index Block: 2
Enter Number of Blocks: 3
Enter Block Numbers:
1
2
3
Index Block : 2
Allocated Blocks : 1 2 3
(kali@thamim72~)$
```

SHELL PROGRAMMING : INDEX



```
GNU nano 9.0 index.sh
#!/bin/bash
echo "Enter Index Block:"
read index
echo "Enter Number of Blocks:"
read n
echo "Enter Block Numbers:"
for ((i=0;i<n;i++))
do
read block[$i]
done
echo "Index Block : $index"
echo -n "Allocated Blocks : "
for ((i=0;i<n;i++))
do
echo -n "${block[$i]} "
done
echo
```

OUTPUT :



```
kali@thamim72:~$ gcc index.c -o inindex
(kali@thamim72:~)$ gcc inindex.c -o index
(kali@thamim72:~)$ ./index
Enter Index Block: 2
Enter Number of Blocks: 3
Enter Block Numbers:
1
2
3
Index Block : 2
Allocated Blocks : 1 2 3
(kali@thamim72:~)$ nano inoex.sn
(kali@thamim72:~)$ bash inoex.sn
Enter Index Block:
2
Enter Number of Blocks:
1
Enter Block Numbers:
2
Index Block : 2
Allocated Blocks : 2
(kali@YUVARAJK)-[~]$
```